

Fixing TLS Live Capture GUI

<https://gemini.google.com/app/63ef69efceb53828>

User prompt: Okay so here is the thing, we have this project for TLS fingerprinting, which I will be uploading the folder for. I want you to go over it. I am building the GUI folder, but the problem is that I cannot see any new records showing up when I press the start live capture button. Can you please tell me how I can fix that? what I want you to do is that you need to go over all the files, see how they connect, understand the architecture, reiterate and go over everything again, then you need to see what is going wrong with the GUI.

Response: The lack of new records is caused by the LiveCaptureWorker thread silently crashing in the background. Scapy's AsyncSniffer will throw an exception if it is passed the literal string "Default" as an interface or if the application lacks Administrator/root privileges to open raw sockets. Because these exceptions are not caught, the worker thread dies immediately without notifying the GUI.

1 Add Error Handling to LiveCaptureWorker

code/gui/workers.py

Translate the "Default" fallback to None so Scapy knows to sniff on all interfaces, and wrap the sniffer initialization in a try...except block with a new errorOccurred signal.

```
class LiveCaptureWorker(QThread):
    rowExtracted = pyqtSignal(dict)
    errorOccurred = pyqtSignal(str) # Add this signal

    def __init__(self, interfaceName, dbPath):
        super().__init__()
        self.interfaceName = interfaceName
        self.fingerprintDb = FingerprintDB(str(dbPath))
        self.captureStats = CaptureStats()
        self.reassembler = TCPReassembler(self.captureStats)
        self.sniffer = None
        self.isRunning = True

    # ... keep processLivePacket exactly as is ...

    def run(self):
        try:
            # Scapy expects None to sniff all interfaces, not the literal
```

```

string "Default"
        ifaceArg = None if self.interfaceName == "Default" else
self.interfaceName

        self.sniffer = AsyncSniffer(
            iface=ifaceArg,
            filter="tcp port 443",
            prn=self.processLivePacket,
            store=False
        )
        self.sniffer.start()

        # Check if the sniffer silently dies in the background thread
        while self.isRunning and self.sniffer.running:
            self.msleep(100)

        if self.sniffer and self.sniffer.running:
            self.sniffer.stop()

    except Exception as e:
        self.errorOccurred.emit(f"Capture failed (Run as Admin/Root?):
{e}")

    def stopCapture(self):
        self.isRunning = False
        self.wait()

```

2 Surface the Errors in the GUI

code/gui/window.py

Update toggleLiveCapture to listen for the new error signal. This will push any Scapy startup exceptions directly into your text pane so you can see if the OS is blocking the capture.

```

def toggleLiveCapture(self):
    if self.liveWorker and self.liveWorker.isRunning:
        self.liveWorker.stopCapture()
        self.liveWorker = None
        self.liveButton.setText("Start Live Capture")
        self.loadButton.setEnabled(True)
    else:
        selectedInterface = self.interfaceDropdown.currentText()
        self.dataTable.setRowCount(0)
        self.rowCache.clear()
        self.detailsPane.clear()

        self.liveWorker = LiveCaptureWorker(selectedInterface,
self.dbPath)
        self.liveWorker.rowExtracted.connect(self.appendTableRow)

```

```

# Bind the new error signal to the UI pane
self.liveWorker.errorOccurred.connect(
    lambda err: self.detailsPane.setText(err)
)

self.liveWorker.start()
self.liveButton.setText("Stop Live Capture")
self.loadButton.setEnabled(False)

```

User prompt: wait, no. Tell me exactly what needs to be done. The rest of the pipeline other than the GUI was built by my peers, and atleast the python one seems to be working fine. Tell me exactly what needs to be done.

Response: The lack of new records is caused by the LiveCaptureWorker thread silently crashing in the background. Scapy's AsyncSniffer will throw an exception if it is passed the literal string "Default" as an interface or if the application lacks Administrator/root privileges to open raw sockets. Because these exceptions are not caught, the worker thread dies immediately without notifying the GUI.

code/gui/workers.py Translate the "Default" fallback to None so Scapy knows to sniff on all interfaces, and wrap the sniffer initialization in a try...except block with a new errorOccurred signal.

```

class LiveCaptureWorker(QThread):
    rowExtracted = pyqtSignal(dict)
    errorOccurred = pyqtSignal(str) # Add this signal

    def __init__(self, interfaceName, dbPath):
        super().__init__()
        self.interfaceName = interfaceName
        self.fingerprintDb = FingerprintDB(str(dbPath))
        self.captureStats = CaptureStats()
        self.reassembler = TCPReassembler(self.captureStats)
        self.sniffer = None
        self.isRunning = True

    # ... keep processLivePacket exactly as is ...

    def run(self):
        try:
            # Scapy expects None to sniff all interfaces, not the literal
            string "Default"
            ifaceArg = None if self.interfaceName == "Default" else
            self.interfaceName

            self.sniffer = AsyncSniffer(
                iface=ifaceArg,
                filter="tcp port 443",
                prn=self.processLivePacket,
                store=False
            )

```

```

        self.sniffer.start()

        while self.isRunning:
            self.msleep(100)

        if self.sniffer and self.sniffer.running:
            self.sniffer.stop()

    except Exception as exceptionObject:
        self.errorOccurred.emit(f"Capture failed (Run as Admin/Root?): {exceptionObject}")

    def stopCapture(self):
        self.isRunning = False
        self.wait()

```

code/gui/window.py Update toggleLiveCapture to listen for the new error signal. This will push any Scapy startup exceptions directly into your text pane so you can see if the OS is blocking the capture.

```

def toggleLiveCapture(self):
    if self.liveWorker and self.liveWorker.isRunning:
        self.liveWorker.stopCapture()
        self.liveWorker = None
        self.liveButton.setText("Start Live Capture")
        self.loadButton.setEnabled(True)
    else:
        selectedInterface = self.interfaceDropdown.currentText()
        self.dataTable.setRowCount(0)
        self.rowCache.clear()
        self.detailsPane.clear()

        self.liveWorker = LiveCaptureWorker(selectedInterface, self.dbPath)
        self.liveWorker.rowExtracted.connect(self.appendTableRow)

        # Bind the new error signal to the UI pane
        self.liveWorker.errorOccurred.connect(
            lambda errorMsg: self.detailsPane.setText(errorMsg)
        )

        self.liveWorker.start()
        self.liveButton.setText("Stop Live Capture")
        self.loadButton.setEnabled(False)

```

Architecture & Connection Flow The project relies on a rigid separation of concerns that allows the GUI and CLI to share the same underlying stream engine:

- **Packet Capture & Reassembly (capture.py):** For offline PCAPs, read_pcap uses dpkt to parse the link and network layers, feeding the TCP segments into TCPReassembler. The reassembler manages flow state (SYN/FIN/RST tracking), resolves out-of-order buffering, and extracts complete TLS

records.

- **TLS Parsing (parser.py & ja3.py):** TCPReassembler passes raw Handshake bodies to parser.py, which maps the raw bytes into ClientHelloData or ServerHelloData structures while preserving wire order. ja3.py then filters out GREASE values and computes the MD5 hashes.
 - **Database (db.py):** A lightweight JSON key-value store matching the computed JA3/JA3S hashes to known clients.
 - **GUI Layer (window.py & workers.py):** To prevent blocking the PyQt6 event loop, heavy I/O is pushed to QThreads. PcapWorker leverages the native dpkt flow. LiveCaptureWorker bypasses dpkt and uses Scapy's AsyncSniffer to capture live packets, extracting the IPs, ports, sequence numbers, and payloads, and manually driving them into the same TCPReassembler used by the offline engine.
-

User prompt: Hmm, which interface? A previous chat told me to confirm with wireshark, but as you can see in the second screenshot, it is hard to decipher.

Response: The active interface with network traffic in image_b77c94.png is **Wi-Fi**, but your Python GUI is displaying the raw Windows Npcap device GUIDs. To find the exact GUID that corresponds to your Wi-Fi adapter:

Find the GUID in Wireshark

- Click the **Manage Interfaces...** button in the bottom right corner of the Wireshark Capture Options window you already have open.
- Select the **Local Interfaces** tab.
- Find the row where the "Friendly Name" is **Wi-Fi**.
- Note the \Device\NPF_{...} string listed in the "Device Name" column for that row.
- Select that exact GUID from your Python GUI dropdown.

Find the GUID via Command Line

- Open your terminal or command prompt.
 - Run the command `getmac /v` to display a list of all network adapters.
 - Match the "Wi-Fi" connection name to its transport name, which contains the GUID you need to select in your GUI.
-

User prompt: Okay so, I think it works, but there are two problems. 1) I feel like we should display friendly names. 2) Matched Client is always unknown. Is this some problem with the DB. I want you to take a deep look and tell me what needs to be done.

Response: Your application is actually functioning perfectly, but you need to swap Scapy's interface enumeration method to show friendly names, and test with non-randomizing clients to see JA3 matches.

1. Displaying Friendly Interface Names Scapy's `get_if_list()` only returns the raw Npcap GUIDs on Windows. You can swap this for `get_working_ifaces()` to populate the dropdown. This allows you to display the human-readable adapter name while storing the raw Npcap GUID as hidden data for the LiveCaptureWorker to use.

Update code/gui/window.py with the following changes:

```

# Change the import at the top
from scapy.all import get_working_ifaces

# ... inside __init__ ...
self.interfaceDropdown = QComboBox()
try:
    # Iterate through actual interface objects
    for iface in get_working_ifaces():
        # Parameter 1: Display Text ("Wi-Fi")
        # Parameter 2: User Data ("\Device\NPF_{...}")
        self.interfaceDropdown.addItem(iface.name, iface.network_name)
except Exception:
    self.interfaceDropdown.addItem("Default", "Default")
controlLayout.addWidget(self.interfaceDropdown)

# ... inside toggleLiveCapture ...
def toggleLiveCapture(self):
    if self.liveWorker and self.liveWorker.isRunning:
        # ... (keep existing stop logic)
    else:
        # Retrieve the hidden GUID instead of the display text
        selectedInterface = self.interfaceDropdown.currentData()
        self.dataTable.setRowCount(0)
        # ... (keep remaining start logic)

```

2. The "Unknown" Client Mystery (TLS Fingerprint Randomization) Your database and parsing logic are not broken. The reason you only see "Unknown" for web browsers is due to **TLS ClientHello Extension Randomization** (a privacy feature recently introduced by Google Chrome and adopted by Firefox and Brave).

- **The JA3 Flaw:** JA3 calculates its MD5 hash based on the *exact sequence* of extensions in the ClientHello packet.
- **The Evasion:** Modern browsers now deliberately shuffle the order of these extensions on every single request. Consequently, the browser generates a completely unique JA3 hash every time you refresh a page, making static database lookups impossible. Your README.md explicitly notes this as a targeted limitation to demonstrate for the project.
- **The Fix:** To verify your database lookup works, you must test against a client that does not randomize its extensions. Open a terminal and run `curl [https://example.com](https://example.com)` while your live capture is running. Your database contains the static hash for modern curl (7a73fb...), and it will successfully populate the "Matched Client" column. You can also run your provided `requests_client.py` script to see a static Python TLS footprint register in the GUI.

User prompt: You need to look at the Matched Client issue again. Because I made this curl req `curl.exe --trace tls_dump.txt https://cloudflare.com` and that still didnt show up as a matched client. Is it because somehow I didnt have a db setup on my machine or something? Remember, I just cloned the repo and wrote code on top of it.

Response: The database file is already present and actively loaded by your GUI from `code/python/fingerprints.json`. The mismatch is happening because JA3 fingerprints the underlying TLS

library, not the application name.

- **The OS TLS Difference:** The curl fingerprint currently stored in your database (7a73fb0bdeaa2a79fadf356c34e7577e) was captured from a Linux environment using OpenSSL.
- **Schannel vs. OpenSSL:** Because you ran curl.exe on Windows, the request was built using Microsoft's native Schannel TLS provider instead of OpenSSL. This means the cipher suites and extensions are fundamentally different, resulting in a completely different JA3 hash that the Linux-based reference database does not recognize.

To fix this, you need to add your specific Windows fingerprint to the local reference database.

1 Identify the Windows Hash

Click the "Unknown" curl.exe row in your GUI. The bottom text pane will display the calculated JA3 hash for the Windows request.

2 Update the Reference List

Open `code/reference/expected_hashes.csv` and add your newly discovered hash to the bottom of the list: `YOUR_NEW_HASH,curl (Windows Schannel)`

3 Rebuild the JSON Database

Run the seed script from the terminal to merge the CSV updates into the JSON database: `python code/db/seed_reference_db.py`

User prompt: Uploading the updated code folder. We need to fix the following things in the UI: 1) Need a way to clear all records from the UI. A button on the top to clear it. 2) Option to save the captured packets as a pcap file 3) Currently you can click fields and edit them with your keyboard lol. We dont want that

Response: To implement the Clear button, PCAP export functionality, and disable in-place cell editing, update `gui/workers.py` and `gui/window.py`.

1. Update `gui/workers.py`

Modify `LiveCaptureWorker` to accept a `packetBuffer` list and buffer every raw Scapy packet intercepted

during live capture.

```
import os
from PyQt6.QtCore import QThread, pyqtSignal
from scapy.all import AsyncSniffer, IP, IPv6, TCP
from src.capture import read_pcap, TCPReassembler, CaptureStats
from src.ja3 import compute_ja3_hash, compute_ja3_string, compute_ja3s_hash
from src.ja4 import compute_ja4_string, compute_ja4s_string
from src.db import FingerprintDB

def client_packet_data(result, client_hello, fingerprint_db):
    ja3_hash = compute_ja3_hash(client_hello)
    ja4_hash = compute_ja4_string(client_hello)
    ja3_match = fingerprint_db.lookup(ja3_hash, "ja3") or "Unknown"
    ja4_match = fingerprint_db.lookup(ja4_hash, "ja4") or "Unknown"
    target_kind = "ja3" if ja3_match == "Unknown" else "ja4"
    target_hash = ja3_hash if target_kind == "ja3" else ja4_hash
    return {
        "source": f"{result.src_ip}:{result.src_port}",
        "destination": f"{result.dst_ip}:{result.dst_port}",
        "sni": client_hello.server_name or "N/A",
        "ja3Hash": ja3_hash,
        "ja4Hash": ja4_hash,
        "ja3Match": ja3_match,
        "ja4Match": ja4_match,
        "fingerprintKind": target_kind,
        "fingerprintHash": target_hash,
        "ja3Raw": compute_ja3_string(client_hello),
    }

def server_packet_data(result, server_hello, fingerprint_db):
    ja3s_hash = compute_ja3s_hash(server_hello)
    ja4s_hash = compute_ja4s_string(server_hello)
    ja3s_match = fingerprint_db.lookup(ja3s_hash, "ja3s") or "Unknown"
    ja4s_match = fingerprint_db.lookup(ja4s_hash, "ja4s") or "Unknown"
    target_kind = "ja3s" if ja3s_match == "Unknown" else "ja4s"
    target_hash = ja3s_hash if target_kind == "ja3s" else ja4s_hash
    return {
        "source": f"{result.src_ip}:{result.src_port}",
        "destination": f"{result.dst_ip}:{result.dst_port}",
        "sni": "N/A",
        "ja3Hash": ja3s_hash,
        "ja4Hash": ja4s_hash,
        "ja3Match": ja3s_match,
        "ja4Match": ja4s_match,
        "fingerprintKind": target_kind,
        "fingerprintHash": target_hash,
        "ja3Raw": "N/A",
    }

class PcapWorker(QThread):
    rowExtracted = pyqtSignal(dict)
```

```

captureFinished = pyqtSignal(str)
errorOccurred = pyqtSignal(str)

def __init__(self, pcapPath, dbPath):
    super().__init__()
    self.pcapPath = pcapPath
    self.fingerprintDb = FingerprintDB(str(dbPath))
    self.captureStats = CaptureStats()

def run(self):
    try:
        if not os.path.exists(self.pcapPath) or
os.path.getsize(self.pcapPath) < 24:
            self.errorOccurred.emit("Selected file is empty or corrupted.")
            return

        for result in read_pcap(self.pcapPath, self.captureStats):
            if result.client_hello:
                self.rowExtracted.emit(
                    client_packet_data(result, result.client_hello,
self.fingerprintDb)
                )
            if result.server_hello:
                self.rowExtracted.emit(
                    server_packet_data(result, result.server_hello,
self.fingerprintDb)
                )
            self.captureFinished.emit(self.captureStats.summary())
    except Exception as e:
        self.errorOccurred.emit(str(e))

class LiveCaptureWorker(QThread):
    rowExtracted = pyqtSignal(dict)
    errorOccurred = pyqtSignal(str)

    def __init__(self, interfaceName, dbPath, packetBuffer=None):
        super().__init__()
        self.interfaceName = interfaceName
        self.fingerprintDb = FingerprintDB(str(dbPath))
        self.captureStats = CaptureStats()
        self.reassembler = TCPReassembler(self.captureStats)
        self.sniffer = None
        self.isRunning = True
        self.packetBuffer = packetBuffer

    def processLivePacket(self, pkt):
        if self.packetBuffer is not None:
            self.packetBuffer.append(pkt)

        if not (pkt.haslayer(TCP) and (pkt.haslayer(IP) or
pkt.haslayer(IPv6))):
            return

```

```

ipLayer = pkt[IP] if pkt.haslayer(IP) else pkt[IPv6]
tcpLayer = pkt[TCP]
srcIp = str(ipLayer.src)
dstIp = str(ipLayer.dst)
srcPort = int(tcpLayer.sport)
dstPort = int(tcpLayer.dport)
seq = int(tcpLayer.seq)
flags = int(tcpLayer.flags)
payload = bytes(tcpLayer.payload)
timestamp = float(pkt.time)

results = self.reassembler.process_packet(
    srcIp, dstIp, srcPort, dstPort, seq, flags, payload, timestamp
)
for res in results:
    if res.client_hello:
        self.rowExtracted.emit(
            client_packet_data(res, res.client_hello,
self.fingerprintDb)
        )
    if res.server_hello:
        self.rowExtracted.emit(
            server_packet_data(res, res.server_hello,
self.fingerprintDb)
        )

def run(self):
    try:
        ifaceArg = None if self.interfaceName == "Default" else
self.interfaceName
        self.sniffer = AsyncSniffer(
            iface=ifaceArg,
            filter="tcp port 443",
            prn=self.processLivePacket,
            store=False
        )
        self.sniffer.start()

        while self.isRunning:
            self.msleep(100)

            if self.sniffer and self.sniffer.running:
                self.sniffer.stop()
        except Exception as exceptionObject:
            self.errorOccurred.emit(f"Capture failed (Run as Admin/Root?):
{exceptionObject}")

def stopCapture(self):
    self.isRunning = False
    self.wait()

```

2. Update gui/window.py

In window.py:

- Add Clear Records and Save PCAP buttons to the top control layout.
- Set `setEditTriggers(QAbstractItemView.EditTrigger.NoEditTriggers)` on `dataTable` and remove the `ItemsEditable` flag on every item so keyboard input and double clicks cannot edit cell text.
- Track loaded PCAPs and live packet buffers, flushing them to disk via `wrpcap` when requested.

```
from pathlib import Path
import shutil
from PyQt6.QtCore import Qt
from PyQt6.QtWidgets import (
    QMainWindow, QWidget, QVBoxLayout, QHBoxLayout, QPushButton,
    QTableWidgetItem, QTableWidgetItem, QFileDialog, QHeaderView,
    QComboBox, QLabel, QTextEdit, QInputDialog, QAbstractItemView
)
from scapy.all import get_working_ifaces, wrpcap
from src.db import FingerprintDB
from workers import PcapWorker, LiveCaptureWorker

class TlsMonitorGui(QMainWindow):
    def __init__(self, pythonEngineDir):
        super().__init__()
        self.setWindowTitle("TLS Fingerprint Monitor")
        self.resize(1150, 650)
        self.dbPath = pythonEngineDir / "fingerprints.json"
        self.pcapsDir = pythonEngineDir.parent / "pcaps"
        self.fingerprintDb = FingerprintDB(str(self.dbPath))
        self.pcapWorker = None
        self.liveWorker = None
        self.rowCache = []
        self.capturedPackets = []
        self.loadedPcapPath = None

        mainWidget = QWidget()
        self.setCentralWidget(mainWidget)
        mainLayout = QVBoxLayout(mainWidget)

        controlLayout = QHBoxLayout()

        self.loadButton = QPushButton("Load PCAP File")
        self.loadButton.clicked.connect(self.selectPcapFile)
        controlLayout.addWidget(self.loadButton)

        self.saveButton = QPushButton("Save PCAP")
        self.saveButton.clicked.connect(self.savePcapFile)
        controlLayout.addWidget(self.saveButton)

        self.clearButton = QPushButton("Clear Records")
```

```

self.clearButton.clicked.connect(self.clearRecords)
controlLayout.addWidget(self.clearButton)

controlLayout.addWidget(QLabel("Interface:"))
self.interfaceDropdown = QComboBox()
try:
    for iface in get_working_ifaces():
        self.interfaceDropdown.addItem(iface.name, iface.network_name)
except Exception:
    self.interfaceDropdown.addItem("Default", "Default")
controlLayout.addWidget(self.interfaceDropdown)

self.liveButton = QPushButton("Start Live Capture")
self.liveButton.clicked.connect(self.toggleLiveCapture)
controlLayout.addWidget(self.liveButton)

self.labelButton = QPushButton("Label Selected Fingerprint")
self.labelButton.clicked.connect(self.labelSelectedFingerprint)
self.labelButton.setEnabled(False)
controlLayout.addWidget(self.labelButton)

controlLayout.addStretch()
mainLayout.addLayout(controlLayout)

self.dataTable = QTableWidget(0, 7)
self.dataTable.setHorizontalHeaderLabels([
    "Source", "Destination", "SNI", "JA3 Hash", "JA4 Hash",
    "JA3 Match", "JA4 Match"
])

self.dataTable.horizontalHeader().setSectionResizeMode(QHeaderView.ResizeMode.S
tretch)

self.dataTable.setSelectionBehavior(QAbstractItemView.SelectionBehavior.SelectR
ows)

self.dataTable.setEditTriggers(QAbstractItemView.EditTrigger.NoEditTriggers)
self.dataTable.itemSelectionChanged.connect(self.displayRowDetails)
mainLayout.addWidget(self.dataTable)

self.detailsPane = QTextEdit()
self.detailsPane.setReadOnly(True)
self.detailsPane.setFixedHeight(120)
self.detailsPane.setPlaceholderText("Select any handshake above to view
raw metadata and JA3 string.")
mainLayout.addWidget(self.detailsPane)

def selectPcapFile(self):
    filePath, _ = QFileDialog.getOpenFileName(
        self, "Open PCAP", str(self.pcapsDir), "PCAP Files (*.pcap
*.pcapng)"
    )

```

```

    if filePath:
        self.dataTable.setRowCount(0)
        self.rowCache.clear()
        self.capturedPackets.clear()
        self.loadedPcapPath = filePath
        self.detailsPane.clear()

        self.pcapWorker = PcapWorker(filePath, self.dbPath)
        self.pcapWorker.rowExtracted.connect(self.appendTableRow)
        self.pcapWorker.errorOccurred.connect(lambda err:
self.detailsPane.setText(f"Error: {err}"))
        self.pcapWorker.start()

def toggleLiveCapture(self):
    if self.liveWorker and self.liveWorker.isRunning:
        self.liveWorker.stopCapture()
        self.liveWorker = None
        self.liveButton.setText("Start Live Capture")
        self.loadButton.setEnabled(True)
    else:
        selectedInterface = self.interfaceDropdown.currentData()
        self.dataTable.setRowCount(0)
        self.rowCache.clear()
        self.capturedPackets.clear()
        self.loadedPcapPath = None
        self.detailsPane.clear()

        self.liveWorker = LiveCaptureWorker(
            selectedInterface, self.dbPath,
packetBuffer=self.capturedPackets
        )
        self.liveWorker.rowExtracted.connect(self.appendTableRow)
        self.liveWorker.errorOccurred.connect(
            lambda errorMsg: self.detailsPane.setText(errorMsg)
        )
        self.liveWorker.start()
        self.liveButton.setText("Stop Live Capture")
        self.loadButton.setEnabled(False)

def clearRecords(self):
    self.dataTable.setRowCount(0)
    self.rowCache.clear()
    self.capturedPackets.clear()
    self.loadedPcapPath = None
    self.detailsPane.clear()
    self.labelButton.setEnabled(False)

def savePcapFile(self):
    if not self.capturedPackets and not self.loadedPcapPath:
        self.detailsPane.setText("No captured or loaded packets available
to save.")
    return

```

```

        savePath, _ = QFileDialog.getSaveFileName(
            self, "Save PCAP", str(self.pcapsDir / "captured.pcap"), "PCAP
Files (*.pcap *.pcapng)"
        )
        if not savePath:
            return

        try:
            if self.capturedPackets:
                packetsSnapshot = list(self.capturedPackets)
                wrpcap(savePath, packetsSnapshot)
                self.detailsPane.setText(f"Successfully saved
{len(packetsSnapshot)} packet(s) to {savePath}")
            elif self.loadedPcapPath:
                shutil.copyfile(self.loadedPcapPath, savePath)
                self.detailsPane.setText(f"Successfully exported PCAP to
{savePath}")
            except Exception as err:
                self.detailsPane.setText(f"Failed to save PCAP: {err}")

    def appendTableRow(self, packetData):
        currentRow = self.dataTable.rowCount()
        self.dataTable.insertRow(currentRow)
        self.rowCache.append(packetData)

        columnValues = [
            packetData["source"],
            packetData["destination"],
            packetData["sni"],
            packetData["ja3Hash"],
            packetData["ja4Hash"],
            packetData["ja3Match"],
            packetData["ja4Match"],
        ]

        for colIdx, val in enumerate(columnValues):
            item = QTableWidgetItem(str(val))
            item.setFlags(item.flags() & ~Qt.ItemFlag.ItemIsEditable)
            self.dataTable.setItem(currentRow, colIdx, item)

    def displayRowDetails(self):
        selectedRows = self.dataTable.selectionModel().selectedRows()
        if not selectedRows:
            self.labelButton.setEnabled(False)
            return

        rowIndex = selectedRows[0].row()
        if rowIndex < len(self.rowCache):
            entry = self.rowCache[rowIndex]
            isUnknown = entry.get("ja3Match") == "Unknown" or
entry.get("ja4Match") == "Unknown"

```

```

        self.labelButton.setEnabled(isUnknown)
        detailText = (
            f"SNI:          {entry['sni']}\n"
            f"JA3 Hash:       {entry['ja3Hash']}\n"
            f"JA3 Match:       {entry['ja3Match']}\n"
            f"JA4 Hash:       {entry['ja4Hash']}\n"
            f"JA4 Match:       {entry['ja4Match']}\n"
            f"Raw JA3:         {entry.get('ja3Raw', 'N/A')}"
        )
        self.detailsPane.setText(detailText)

def labelSelectedFingerprint(self):
    selectedRows = self.dataTable.selectionModel().selectedRows()
    if not selectedRows:
        return

    rowIndex = selectedRows[0].row()
    if rowIndex >= len(self.rowCache):
        return

    entry = self.rowCache[rowIndex]
    if entry.get("ja3Match") != "Unknown" and entry.get("ja4Match") !=
"Unknown":
        return

    label, accepted = QInputDialog.getText(
        self,
        "Label Fingerprint",
        "Verified client name:",
    )
    if not accepted or not label.strip():
        return

    try:
        self.fingerprintDb.enroll(
            entry["fingerprintHash"],
            entry.get("fingerprintKind", "ja3"),
            label,
        )
    except (OSError, ValueError) as error:
        self.detailsPane.setText(f"Could not save fingerprint: {error}")
        return

    target_match = "ja3Match" if entry.get("fingerprintKind") == "ja3" else
"ja4Match"
    entry[target_match] = label.strip()
    target_column = 5 if target_match == "ja3Match" else 6

    newItem = QTableWidgetItem(entry[target_match])
    newItem.setFlags(newItem.flags() & ~Qt.ItemFlag.ItemIsEditable)
    self.dataTable.setItem(rowIndex, target_column, newItem)
    self.displayRowDetails()

```

User prompt: Is it possible and easy to add another interface dropdown "Any" which looks for all handshakes across all interfaces

Response: Scapy does not natively support the Linux-style "any" pseudo-interface on Windows via Npcap, but you can achieve identical behavior by passing a dynamically generated list of all active network interfaces to the AsyncSniffer.

code/gui/window.py Add the "Any" option to the dropdown menu before iterating through the available interfaces.

```
# ... existing code ...
    controlLayout.addWidget(QLabel("Interface:"))
    self.interfaceDropdown = QComboBox()

    # Add the 'Any' option as the default first choice
    self.interfaceDropdown.addItem("Any (All Interfaces)", "Any")

    try:
        for iface in get_working_ifaces():
            self.interfaceDropdown.addItem(iface.name, iface.network_name)
    except Exception:
        self.interfaceDropdown.addItem("Default", "Default")
    controlLayout.addWidget(self.interfaceDropdown)
# ... existing code ...
```

code/gui/workers.py Import `get_working_ifaces` at the top of the file, then update the `LiveCaptureWorker` to detect the "Any" selection and supply Scapy with the complete list of interfaces.

```
import os
from PyQt6.QtCore import QThread, pyqtSignal
from scapy.all import AsyncSniffer, IP, IPv6, TCP, get_working_ifaces
# ... existing imports ...

# ... inside LiveCaptureWorker ...
def run(self):
    try:
        if self.interfaceName == "Any":
            # Scapy requires an explicit list to sniff multiple interfaces
            concurrently
                ifaceArg = [iface.network_name for iface in
get_working_ifaces()]
            elif self.interfaceName == "Default":
                ifaceArg = None
            else:
                ifaceArg = self.interfaceName

        self.sniffer = AsyncSniffer(
```

```
        iface=ifaceArg,  
        filter="tcp port 443",  
        prn=self.processLivePacket,  
        store=False  
    )  
    self.sniffer.start()  
  
    while self.isRunning:  
        self.msleep(100)  
  
        if self.sniffer and self.sniffer.running:  
            self.sniffer.stop()  
    except Exception as exceptionObject:  
        self.errorOccurred.emit(f"Capture failed (Run as Admin/Root?):  
{exceptionObject}")
```
